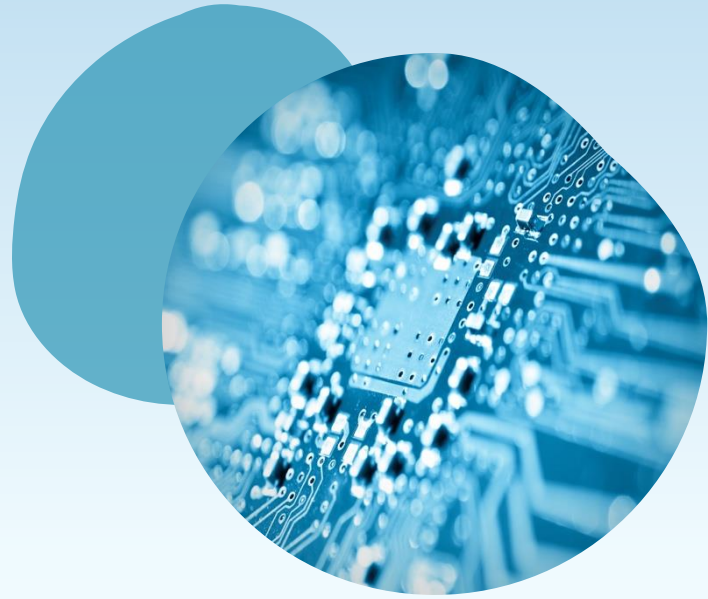




Interrupts

Embedded Software for the Internet Of Things

Kasim Sinan Yildirim

How can we detect when a switch is pressed?

- One option is **polling**:
 - e.g. using software to check it regularly.
- However, polling is:
 - **Slow** – The user needs to check to see if the switch is pressed
 - **Wasteful of CPU time** – the faster the response required, the more often the user needs to check
 - **Not scalable** – Building a multi-activity system that can respond quickly isn't easy.
 - The system's response time depends on all other processing it has to do

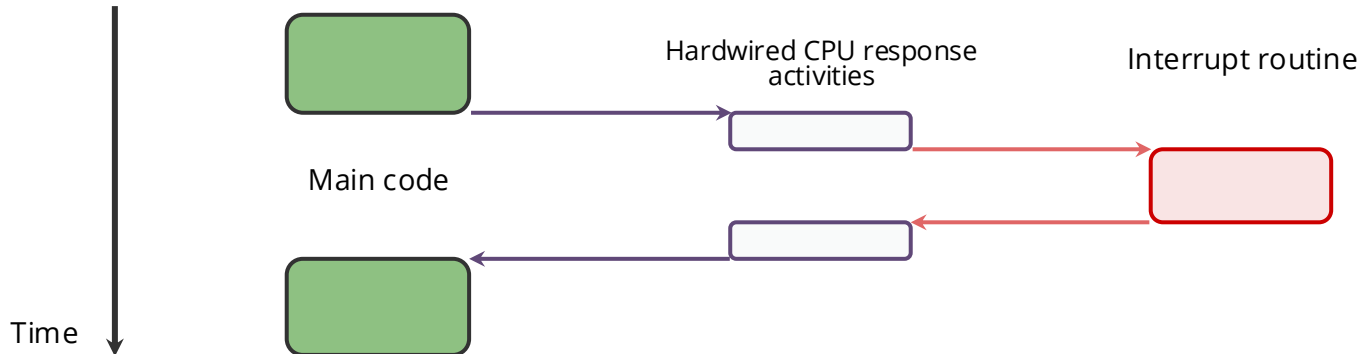
```
...  
while (P1->IN & 2) ;  
while ( ! (P1->IN & 2) ) ;  
...
```

Better Option: Interrupts

- Using special hardware in the MCU to detect and run **Interrupt Service Routine (ISR)** in response.
- An **interrupt** is:
 - **Efficient** – the code runs only when necessary
 - **Fast** – it's a hardware mechanism
 - **Scalable**:
 - ISR response time doesn't depend on most other processing
 - Code modules can be developed independently

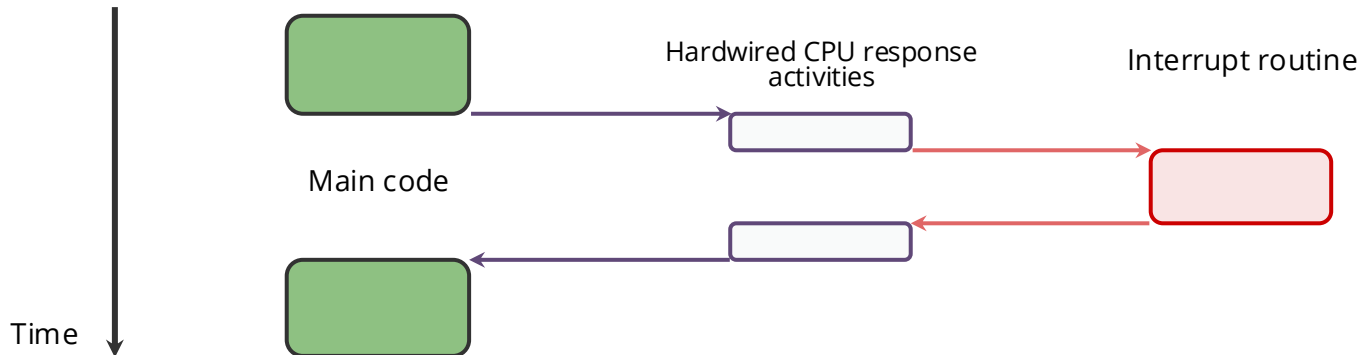
Interrupt Processing Sequence

- Main code is running in the **foreground**.
- When the interrupt trigger occurs:
 - The processor executes the ISR:
 - Returns via the **return-from-interrupt** instruction at the end
 - Similar to the return from functions
 - Then the processor resumes running main code



Interrupt Processing Sequence

- Fundamental mechanism of microcontrollers
 - Provides efficient **event-based processing** rather than polling
 - Provides quick response to events
 - Allows embedded systems to be **responsive** without an operating system

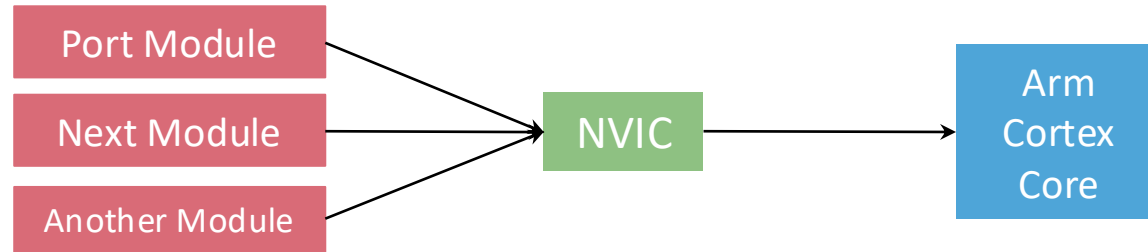


Interrupt Sources

- Hardware-triggered **asynchronous** software routine
 - Triggered by hardware signal from peripheral or external device
 - **Asynchronous** – can happen anywhere in the program (unless interrupt is disabled)
 - E.g., character is received on serial port, or ADC converter finishes conversion
 - Software routine – Interrupt Service Routine runs in response to interrupt
 - After ISR completes, MCU goes back to previously executing code
- Software-triggered interrupts (**exceptions** or **faults**)
 - **Synchronous:** are the result of specific instructions executing
 - Examples: undefined instructions, overflow occurs for a given instruction

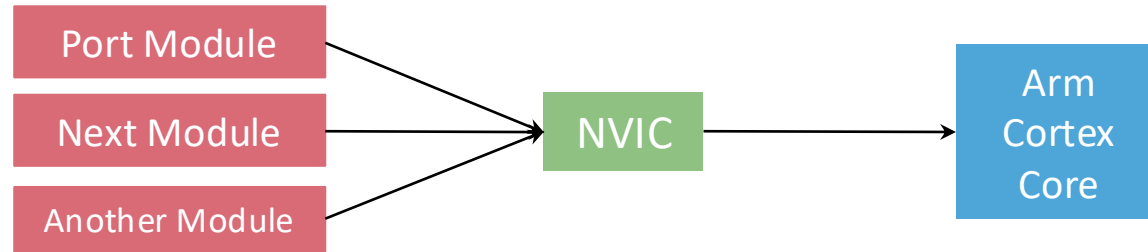
Nested Vectored Interrupt Controller (NVIC)

- Manages and **prioritizes** external interrupts to response simultaneous requests
 - Smaller number -> higher priority



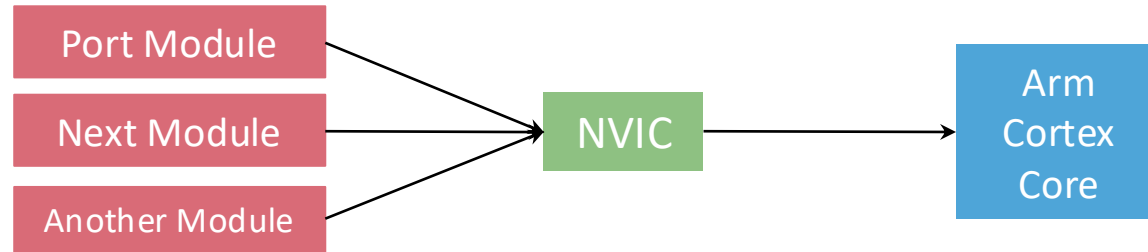
Nested Vectored Interrupt Controller (NVIC)

- Provides **configurable** interrupt handling:
 - Adjustable priorities
 - Priorities of some interrupts are fixed: e.g., **Reset: highest priority**



Nested Vectored Interrupt Controller (NVIC)

- Maskable interrupts:
 - Enable and disable (mask) interrupts as needed
 - There are **non-maskable interrupts (NMI)**



Nested Vectored Interrupt Controller (NVIC)

- Interrupt states: Inactive, Pending, Active, A&P...
- **Pending Interrupts:**
 - An interrupt has been requested and detected but is not serviced yet
 - Possibly, a higher priority interrupt is being processed.
- An interrupt routine call might happen within another active interrupt.
 - When?
 - This second interrupt is called “**nested**”.

Nested Vectored Interrupt Controller (NVIC)

- Simultaneous interrupt requests?
 - High-priority interrupt is serviced first
- New interrupt requested while a handler is executing?
 - New priority higher than current priority?
 - New interrupt handler **preempts** current interrupt handler.
 - New priority lower than or equal to current priority?
 - New interrupt held in **pending state**.
 - Current handler continues and completes execution.
 - Previous priority level restored.
 - New interrupt handled if priority level allows.

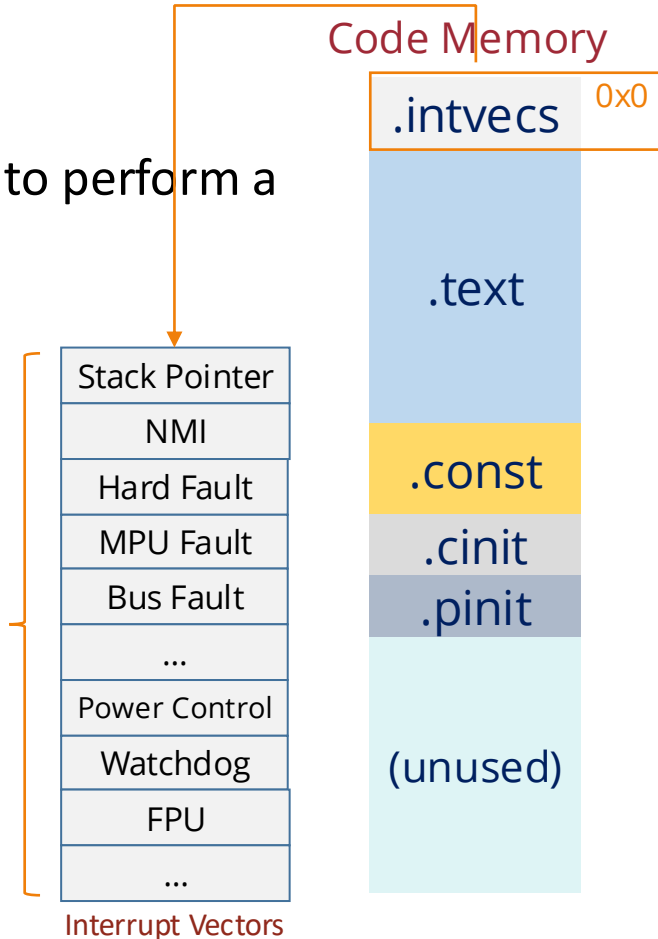
Summary: Interrupt Handling Sequence

- The CPU checks for pending interrupts at the beginning of each instruction:
 - It answers the **highest-priority interrupt**
 - higher priority than that in the interrupt priority register.
- Each device has a dedicated **interrupt vector number**
 - The CPU uses the vector number **as an index** in the **interrupt vector table**
 - gets the device's interrupt handler address.

Interrupt Vector Table

- Interrupts are special events that request the CPU to perform a specific operation
 - E.g. Timers, GPIO, CPU Exception
- **Interrupt Service Routine (ISR)**
 - Function to be called in response to an interrupt request
 - Placed at address 0x0 in code memory (ARM)

Array of
function
addresses



Interrupt Vector Table for Arm Cortex-M

Interrupt #	Interrupt	Memory Location (Hex)
0	Stack Pointer initial value	0x00000000
1	Reset	0x00000004
2	NMI	0x00000008
3	Hard Fault	0x0000000C
4	Memory Management Fault	0x00000010
5	Bus Fault	0x00000014
6	Usage Fault (undefined instructions, divide by zero, unaligned memory access,...)	0x00000018
7	Reserved	0x0000001C
8	Reserved	0x00000020
9	Reserved	0x00000024
10	Reserved	0x00000028
11	SVCall	0x0000002C
12	Debug Monitor	0x00000030
13	Reserved	0x00000034
14	PendSV	0x00000038
15	SysTick	0x0000003C
16	IRQ 0 for peripherals	0x00000040
17	IRQ 1 for peripherals	0x00000044
...	...	...

Interrupt Vector Table for Arm Cortex-M

Interrupt #	Interrupt	Memory Location (Hex)
0	Stack Pointer initial value	SP: 0x00000000
1	Reset	PC: 0x00000004
2	NMI	0x00000008
3	Hard Fault	0x0000000C
4	Memory Management Fault	0x00000010
5	Bus Fault	0x00000014
6	Usage Fault (undefined instructions, divide by zero, unaligned memory access,...)	0x00000018
7	Reserved	0x0000001C
8	Reserved	0x00000020
9	Reserved	0x00000024
10	Reserved	0x00000028
11	SVCall	0x0000002C
12	Debug Monitor	0x00000030
13	Reserved	0x00000034
14	PendSV	0x00000038
15	SysTick	0x0000003C
16	IRQ 0 for peripherals	0x00000040
17	IRQ 1 for peripherals	0x00000044
...	...	...

Interrupt Vector Table for Arm Cortex-M

Interrupt #	Interrupt	Priority Level
0	Stack Pointer initial value	
1	Reset	-3 Highest
2	NMI	-2
3	Hard Fault	-1
4	Memory Management Fault	Programmable
5	Bus Fault	Programmable
6	Usage Fault (undefined instructions, divide by zero, unaligned memory access,...)	Programmable
7	Reserved	Programmable
8	Reserved	Programmable
9	Reserved	Programmable
10	Reserved	Programmable
11	SVCall	Programmable
12	Debug Monitor	Programmable
13	Reserved	Programmable
14	PendSV	Programmable
15	SysTick	Programmable
16	IRQ 0 for peripherals	Programmable
17	IRQ 1 for peripherals	Programmable
...	...	Programmable

Interrupt Vector Table

- Definition requires both **linker mapping** and C/assembly code

Linker Script

```
SECTIONS
{
    .intvecs : > 0x00000000
    .text : > MAIN
    .const : > MAIN
    ...
}
```

MSP432 Startup File

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END), /* Initial stack pointer */
    reset_ISR, /* Reset handler */
    nmi_ISR, /* NMI handler */
    fault_ISR, /* Hard fault handler */
    mpu_ISR, /* MPU fault handler */
    busfault_ISR, /* Bus fault handler */
    ... /* More Interrupt handlers */
}
```

Interrupt Vector Table

- Definition requires both **linker mapping** and C/assembly code

Linker Script

```
SECTIONS
{
    .intvecs : > 0x00000000
    .text : > MAIN
    .const : > MAIN
    ...
}
```

MSP432 Startup File

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END), /* Initial stack pointer */
    reset_ISR, /* Reset handler */
    nmi_ISR, /* NMI handler */
    fault_ISR, /* Hard fault handler */
    mpu_ISR, /* MPU fault handler */
    busfault_ISR, /* Bus fault handler */
    ... /* More Interrupt handlers */
}
```

Interrupt Vector Table

- Vector table is an **array of function addresses**
 - Used to “**jump**” into a routine when interrupt occurs

MSP432 Startup File

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END),
    reset_ISR,
    nmi_ISR,
    fault_ISR,
    mpu_ISR,
    busfault_ISR,
    ... /* More Interrupt handlers */
}
```

Interrupt Vector Table

- Vector table is an **array of function addresses**
 - Used to “**jump**” into a routine when interrupt occurs

MSP432 Startup File

Function pointer declaration

all Interrupt
handlers are void
functions

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END),
    reset_ISR,
    nmi_ISR,
    fault_ISR,
    mpu_ISR,
    busfault_ISR,
    ... /* More Interrupt handlers */
}
```

Interrupt Vector Table

- Vector table is an **array of function addresses**
 - Used to “**jump**” into a routine when interrupt occurs

Array should be
read only at compile

MSP432 Startup File

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END),
    reset_ISR,
    nmi_ISR,
    fault_ISR,
    mpu_ISR,
    busfault_ISR,
    ... /* More Interrupt handlers */
}
```

Interrupt Vector Table

- Vector table is an **array of function addresses**
 - Used to “**jump**” into a routine when interrupt occurs

MSP432 Startup File

First element is the
initial stack pointer
to initialize the Core
CPU Registers

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END),
    reset_ISR,
    nmi_ISR,
    fault_ISR,
    mpu_ISR,
    busfault_ISR,
    ... /* More Interrupt handlers */
}
```

Interrupt Vector Table

- Vector table is an **array of function addresses**
 - Used to “**jump**” into a routine when interrupt occurs

MSP432 Startup File

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    (void (*)(void)) ((uint32_t) &__STACK_END),
    reset_ISR,
    nmi_ISR,
    fault_ISR,
    mpu_ISR,
    busfault_ISR,
    ... /* More Interrupt handlers */
}
```

High Priority ARM
Core Exceptions

Interrupt Vector Table

- Vector table is an **array of function addresses**
 - Used to “**jump**” into a routine when interrupt occurs

MSP432 Startup File

```
#pragma DATA_SECTION(interruptVectors, ".intvecs")
void (* const interruptVectors[]) (void) =
{
    ...
    PORT1_IRQHandler, /* Port1 Interrupt */
    PORT2_IRQHandler, /* Port2 Interrupt */
    PORT3_IRQHandler, /* Port3 Interrupt */
    PORT4_IRQHandler, /* Port4 Interrupt */
    PORT5_IRQHandler, /* Port5 Interrupt */
    PORT6_IRQHandler  /* Port6 Interrupt */
}
```

Handlers for the
GPIO interrupts

Default Handler

MSP432 Startup File

```
...
/* Forward declaration of the default fault handlers. */
void Default_Handler (void) __attribute__((weak));
...
/* This is the code that gets called when the processor receives an unexpected */
/* interrupt. This simply enters an infinite loop, preserving the system state */
/* for examination by a debugger. */
void Default_Handler(void)
{
    /* Fault trap exempt from ULP advisor */
    #pragma diag_push
    #pragma CHECK_ULP("-2.1")

    /* Enter an infinite loop. */
    while(1)
    {
    }

    #pragma diag_pop
}
```

Default Handler

Compiler Directive: Tells the compiler that this function can be overridden in user code.

MSP432 Startup File

```
...
/* Forward declaration of the default fault handlers. */
void Default_Handler (void) __attribute__((weak));
...
/* This is the code that gets called when the processor receives an unexpected */
/* interrupt. This simply enters an infinite loop, preserving the system state */
/* for examination by a debugger. */
void Default_Handler(void)
{
    /* Fault trap exempt from ULP advisor */
    #pragma diag_push
    #pragma CHECK_ULP("-2.1")

    /* Enter an infinite loop. */
    while(1)
    {
    }

    #pragma diag_pop
}
```

Overriding Default Handlers

Tells the compiler that the application layer can choose to implement these functions or not. By default, `Default_Handler` will be called.

MSP432 Startup File

```
...  
extern void PORT1_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT2_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT3_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT4_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT5_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT6_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
...
```



Overriding Default Handlers

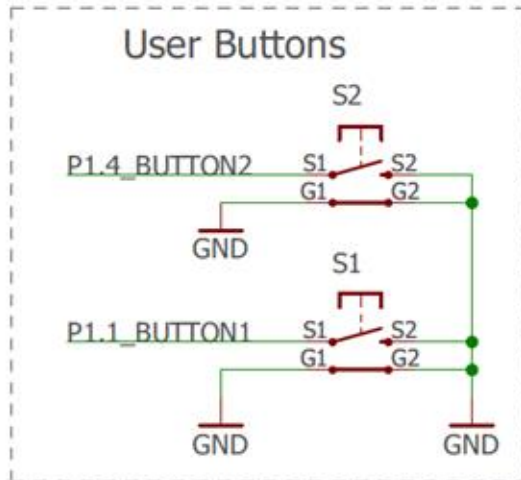
Tells the compiler that the application layer can choose to implement these functions or not. By default, `Default_Handler` will be called.

MSP432 Startup File

```
...  
extern void PORT1_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT2_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT3_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT4_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT5_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
extern void PORT6_IRQHandler (void) __attribute__((weak, alias("Default_Handler")));  
...
```

These functions can be defined in other source files.

MSP432 Launchpad Buttons



Configure User Button (P1.1) - Interrupts

- Interrupt Edge Select Registers (**PxIES**)
 - Interrupt on high-to-low transition

```
P1->IES = BIT1;
```

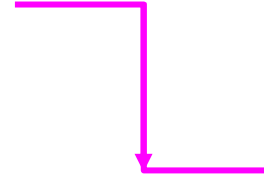


Figure 10-10. PxIES Register

7	6	5	4	3	2	1	0
PxIES							
rw	rw	rw	rw	rw	rw	rw	rw

Table 10-13. P1IES Register Description

Bit	Field	Type	Reset	Description
7-0	PxIES	RW	Undefined	Port X interrupt edge select 0b = PxIFG flag is set with a low-to-high transition. 1b = PxIFG flag is set with a high-to-low transition.

Configure User Button (P1.1) - Interrupts

- Interrupt Enable Registers (**PxIE**)
 - Enable interrupt for P1.1

```
P1->IES = BIT1;  
P1->IE = BIT1;
```

Figure 10-11. PxIE Register

7	6	5	4	3	2	1	0
PxIE							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 10-14. PxIE Register Description

Bit	Field	Type	Reset	Description
7-0	PxIE	RW	0h	Port X interrupt enable 0b = Corresponding port interrupt disabled 1b = Corresponding port interrupt enabled

Configure User Button (P1.1) - Interrupts

- Port X Interrupt Flag Register (**PxIFG**)
 - Clear all P1 interrupt flags

```
P1->IES = BIT1;  
P1->IE = BIT1;  
P1->IFG = 0;
```

Figure 10-12. PxIFG Register

7	6	5	4	3	2	1	0
PxIFG							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 10-15. PxIFG Register Description

Bit	Field	Type	Reset	Description
7-0	PxIFG	RW	0h	Port X interrupt flag 0b = No interrupt is pending. 1b = Interrupt is pending.

NVIC IRQ Assignments in MSP432P401R

INT#	IRQ#	Vector location	Device
1-15	None	0000 0000 to 0000 003C	CPU Exception(set by ARM)
16	0	0000 0040	PSS
17	1	0000 0044	CS
18	2	0000 0048	PCM
19	3	0000 004C	WDT_A
20	4	0000 0050	FPU_INT
21	5	0000 0054	Flash Controller
22	6	0000 0058	COMP_E1
23	7	0000 005C	COMP_E2
24	8	0000 0060	TIMERA0
25	9	0000 0064	TIMERA0
26	10	0000 0068	TIMERA1
27	11	0000 006C	TIMERA1
28	12	0000 0070	TIMERA2

NVIC IRQ Assignments in MSP432P401R

INT#	IRQ#	Vector location	Device
29	13	0000 0074	TIMERA2
30	14	0000 0078	TIMERA3
31	15	0000 007C	TIMERA3
32	16	0000 0080	eUSCI_A0
33	17	0000 0084	eUSCI_A1
34	18	0000 0088	eUSCI_A2
35	19	0000 008C	eUSCI_A3
36	20	0000 0090	eUSCI_B0
37	21	0000 0094	eUSCI_B1
38	22	0000 0098	eUSCI_B3
39	23	0000 009C	eUSCI_B4
40	24	0000 00A0	ADC14
41	25	0000 00A4	TIMER32_INT1
42	26	0000 00A8	TIMER32_INT2

NVIC IRQ Assignments in MSP432P401R

INT#	IRQ#	Vector location	Device
43	27	0000 00AC	TIMER32_INTC
44	28	0000 00B0	AES256
45	29	0000 00B4	RTC_C
46	30	0000 00B8	DMA_ERR
47	31	0000-00BC	DMA_INT3
48	32	0000 00C0	DMA_INT2
49	33	0000 00C4	DMA_INT1
50	34	0000 00C8	DMA_INT0
51	35	0000-00CC	I/O Port P1
52	36	0000 00D0	I/O Port P2
53	37	0000 00D4	I/O Port P3
54	38	0000 00D8	I/O Port P4
55	39	0000-00DC	I/O Port P5
56	40	0000 00E0	I/O Port P6
57-79	41-63	0000 00E4 – 0000 013C	reserved

Nested Vectored Interrupt Controller (NVIC) Registers

- ISER0 and ISER1

Table 2-25. NVIC Registers

Offset	Acronym	Register Name	Type	Reset	Section
100h	ISER0	Irq 0 to 31 Set Enable Register	read-write	00000000h	Section 2.4.3.1
104h	ISER1	Irq 32 to 63 Set Enable Register	read-write	00000000h	Section 2.4.3.2
180h	ICER0	Irq 0 to 31 Clear Enable Register	read-write	00000000h	Section 2.4.3.3
184h	ICER1	Irq 32 to 63 Clear Enable Register	read-write	00000000h	Section 2.4.3.4
200h	ISPR0	Irq 0 to 31 Set Pending Register	read-write	00000000h	Section 2.4.3.5
204h	ISPR1	Irq 32 to 63 Set Pending Register	read-write	00000000h	Section 2.4.3.6
280h	ICPR0	Irq 0 to 31 Clear Pending Register	read-write	00000000h	Section 2.4.3.7
284h	ICPR1	Irq 32 to 63 Clear Pending Register	read-write	00000000h	Section 2.4.3.8
300h	IABR0	Irq 0 to 31 Active Bit Register	read-only	00000000h	Section 2.4.3.9
304h	IABR1	Irq 32 to 63 Active Bit Register	read-only	00000000h	Section 2.4.3.10
400h	IPR0	Irq 0 to 3 Priority Register	read-write	00000000h	Section 2.4.3.11
404h	IPR1	Irq 4 to 7 Priority Register	read-write	00000000h	Section 2.4.3.12
408h	IPR2	Irq 8 to 11 Priority Register	read-write	00000000h	Section 2.4.3.13
40Ch	IPR3	Irq 12 to 15 Priority Register	read-write	00000000h	Section 2.4.3.14
410h	IPR4	Irq 16 to 19 Priority Register	read-write	00000000h	Section 2.4.3.15
414h	IPR5	Irq 20 to 23 Priority Register	read-write	00000000h	Section 2.4.3.16
418h	IPR6	Irq 24 to 27 Priority Register	read-write	00000000h	Section 2.4.3.17
41Ch	IPR7	Irq 28 to 31 Priority Register	read-write	00000000h	Section 2.4.3.18
420h	IPR8	Irq 32 to 35 Priority Register	read-write	00000000h	Section 2.4.3.19
424h	IPR9	Irq 36 to 39 Priority Register	read-write	00000000h	Section 2.4.3.20
428h	IPR10	Irq 40 to 43 Priority Register	read-write	00000000h	Section 2.4.3.21
42Ch	IPR11	Irq 44 to 47 Priority Register	read-write	00000000h	Section 2.4.3.22
430h	IPR12	Irq 48 to 51 Priority Register	read-write	00000000h	Section 2.4.3.23
434h	IPR13	Irq 52 to 55 Priority Register	read-write	00000000h	Section 2.4.3.24
438h	IPR14	Irq 56 to 59 Priority Register	read-write	00000000h	Section 2.4.3.25
43Ch	IPR15	Irq 60 to 63 Priority Register	read-write	00000000h	Section 2.4.3.26
F00h	STIR	Software Trigger Interrupt Register	write-only	00000000h	Section 2.4.3.27

Nested Vectored Interrupt Controller (NVIC) Registers

- Enable Port 1 interrupt
 - Enable **IRQ35**

```
P1->IES = BIT1;  
P1->IE = BIT1;  
P1->IFG = 0;  
NVIC->ISER[1] = 1 << ((PORT1_IRQn) & 31);
```

2.4.3.2 ISER1 Register (Offset = 104h) [reset = 00000000h]

ISER1 is shown in [Figure 2-21](#) and described in [Table 2-27](#).

Irq 32 to 63 Set Enable Register. Use the Interrupt Set-Enable Registers to enable interrupts and determine which interrupts are currently enabled.

Figure 2-21. ISER1 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETENA																															
R/W-0h																															

Table 2-27. ISER1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	SETENA	R/W	0h	Writing 0 to a SETENA bit has no effect, writing 1 to a bit enables the corresponding interrupt. Reading the bit returns its current enable state. Reset clears the SETENA fields.

Nested Vectored Interrupt Controller (NVIC) Registers

- Enable Port 1 interrupt
 - Enable **IRQ35**

```
P1->IES = BIT1;  
P1->IE = BIT1;  
P1->IFG = 0;  
NVIC->ISER[1] = 1 << ((PORT1_IRQn) & 31);
```

2.4.3.2 ISER1 Register (Offset = 104h) [reset = 00000000h]

ISER1 is shown in [Figure 2-21](#) and described in [Table 2-27](#).

Irq 32 to 63 Set Enable Register. Use the Interrupt Set-Enable Registers to enable interrupts and determine which interrupts are currently enabled.

Figure 2-21. ISER1 Register



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETENA																															
R/W-0h																															

Table 2-27. ISER1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	SETENA	R/W	0h	Writing 0 to a SETENA bit has no effect, writing 1 to a bit enables the corresponding interrupt. Reading the bit returns its current enable state. Reset clears the SETENA fields.

Nested Vectored Interrupt Controller (NVIC) Registers

- Enable Port 1 interrupt
 - Enable **IRQ35**

```
P1->IES = BIT1;
P1->IE = BIT1;
P1->IFG = 0;
NVIC->ISER[1] = 1 << ((PORT1_IRQn) & 31);
```

31 = 11111b
PORT1_IRQn = 35 = 100011b

2.4.3.2 ISER1 Register (Offset = 104h) [reset = 00000000h]

ISER1 is shown in Figure 2-21 and described in Table 2-27.

Irq 32 to 63 Set Enable Register. Use the Interrupt Set-Enable Registers to enable interrupts and determine which interrupts are currently enabled.

Figure 2-21. ISER1 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETENA																															
R/W-0h																															

IRQ35

Table 2-27. ISER1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	SETENA	R/W	0h	Writing 0 to a SETENA bit has no effect, writing 1 to a bit enables the corresponding interrupt. Reading the bit returns its current enable state. Reset clears the SETENA fields.

Interrupt Handler

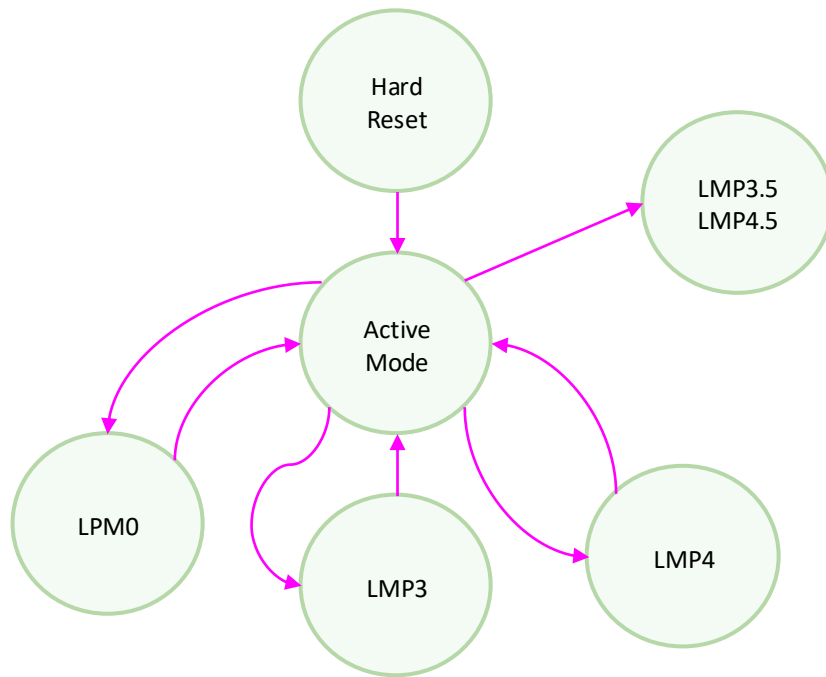
```
/* Port1 ISR */
void PORT1_IRQHandler(void)
{
    // Toggling the output on the LED
    if(P1->IFG & BIT1)
        P1->OUT ^= BIT0;

    // clear the flag
    P1->IFG &= ~BIT1;
}
```

MSP432 Low-Power Modes

- MSP432 supports several power modes for operation:
 - allow for the optimization of power for a given application scenario.

MSP432	Description	Comments	Current
Active	Active Mode	CPU & peripherals	100uA/MHz
LPM0	Sleep	Peri. on, CPU off	65uA/MHz
LPM3	Deepsleep, Stand-by w/ RAM & RTC	some peripherals available	<900nA
LPM4	Stand-by with RAM	No clocks, some peripherals available	<900nA
LPM3.5	Shutdown	RTC w/o RAM	<670nA
LPM4.5	Shutdown	Shutdown	<100nA



MSP432 Low-Power Modes

- MSP432 supports several power modes for operation:
 - allow for the optimization of power for a given application scenario.

MSP432	Description	Comments	Current
Active	Active Mode	CPU & peripherals	100uA/MHz
LPM0	Sleep	Peri. on, CPU off	65uA/MHz
LPM3	Deepsleep, Stand-by w/ RAM & RTC	some peripherals available	<900nA
LPM4	Stand-by with RAM	No clocks, some peripherals available	<900nA
LPM3.5	Shutdown	RTC w/o RAM	<670nA
LPM4.5	Shutdown	Shutdown	<100nA

To enter LPM0 mode, simply use:

```
__sleep() ;
```

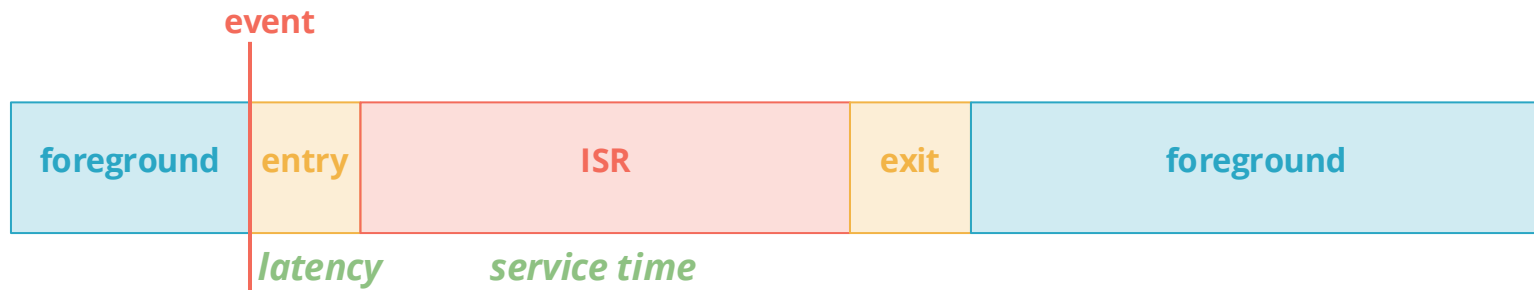
Lab#0

- Let's observe the contents of the registers during ISR handling.
 - Put breakpoint at **PORT1_IRQHandler**

Finish Current Instruction

- Most instructions are short and finish quickly.
- Some instructions may take **many cycles** to execute.
 - Load Multiple (LDM), Store Multiple (STM), Push, Pop, MULS (32 cycles for some CPU core implementations)
- This will delay interrupt response significantly.
- If one of these is executing when the interrupt is requested, the processor:
 - abandons the instruction and responds to the interrupt
 - executes the ISR
 - returns from interrupt and restarts the abandoned instruction

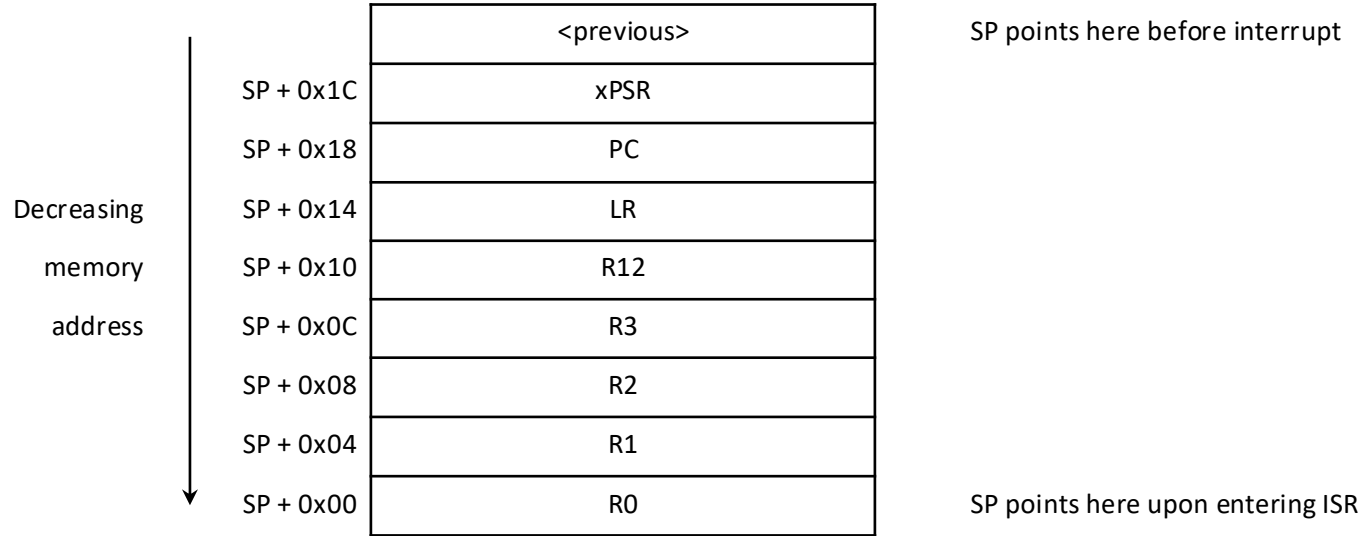
Interrupt timing has distinct components



$$\text{response time} = \text{latency} + \text{service time}$$

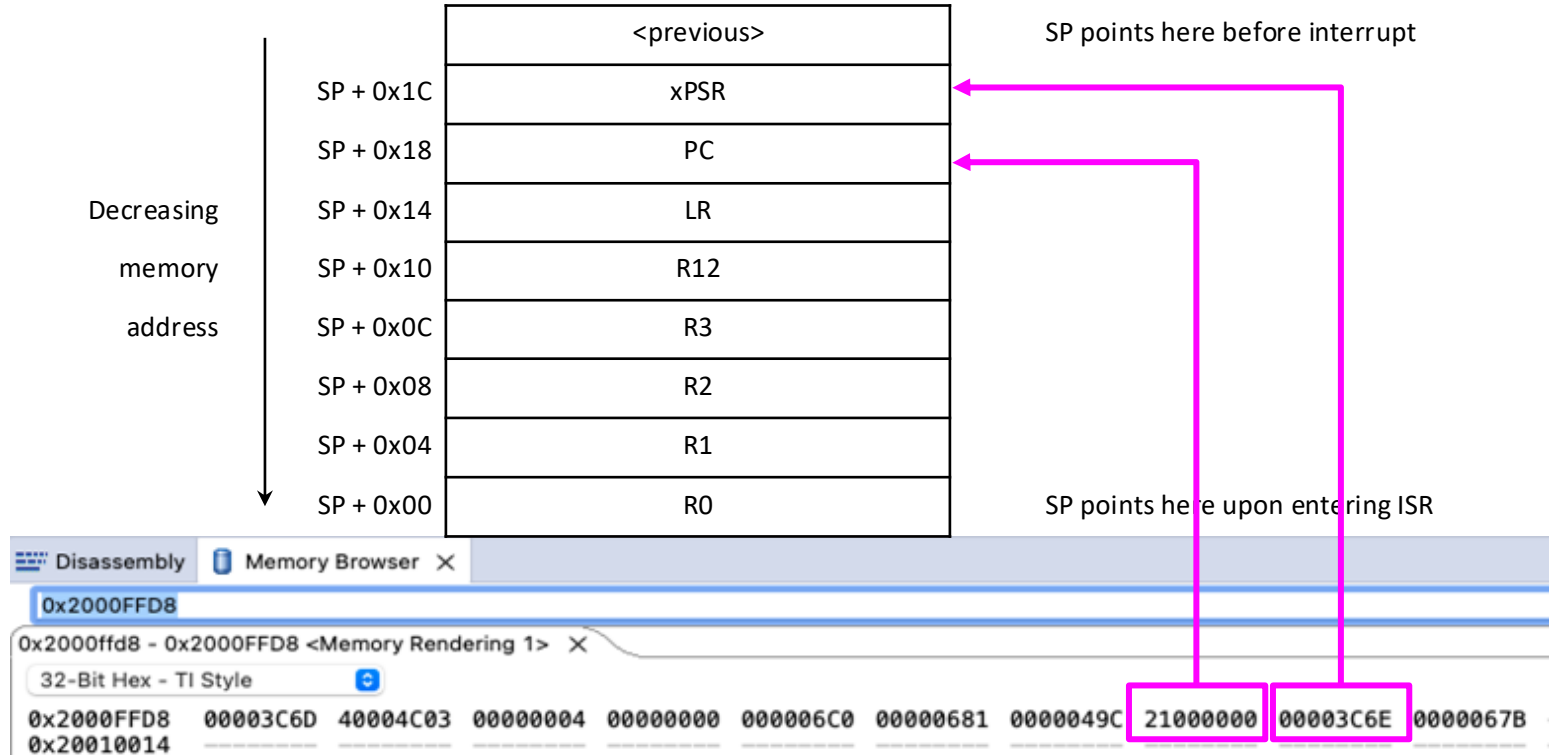
Push Content Onto the Stack

- Before ISR, the CPU pushes **the state** on the stack automatically:



Push Content Onto the Stack

- Before ISR, the CPU pushes **the state** on the stack automatically:



Interrupt Source

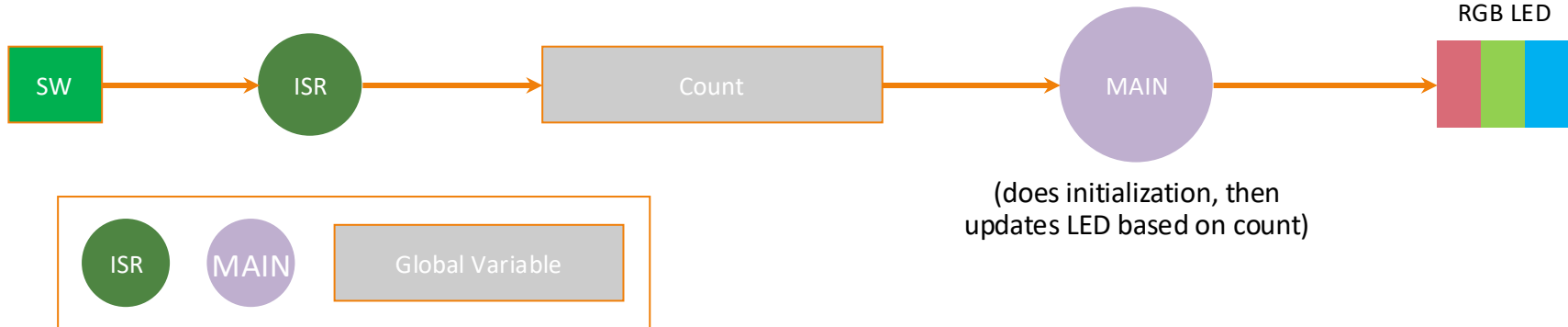
Core Registers		Core Registers	
PC	0x000004A6	Program Counter [Core]	
SP	0x2000FFD8	General Purpose Register 13 - Stack Pointer [Core]	
LR	0xFFFFFFFF9	General Purpose Register 14 - Link Register [Core]	
xPSR	0x21000033	Stores the status of interrupt enables and critical processor status	
N	0	Stores bit 31 of the result of the instruction. In other words stores	
Z	0	Is set to 1 if the result of the operation is zero else stays 0	
C	1	Stores the value of the carry bit if it occurred in an addition or the	
V	0	Set to 1 if an overflow occurred	
Q	0	Indicates whether an overflow/saturation occurred in the enhanced	
ICL_IT_2	00	ICI/IT - bit26-bit25	
T	1	Thumb State	
RESV	00000000	Reserved	
ICL_IT_1	000000	ICI/IT - bit15-bit10	
RESV2	0	Reserved	
EXCEPTION	000110011	Exception Number	

Lab#1

- Write the code that performs the following operation:
 - When SW1 is pressed -> Toggle Red LED
 - When SW2 is pressed -> Toggle Green LED

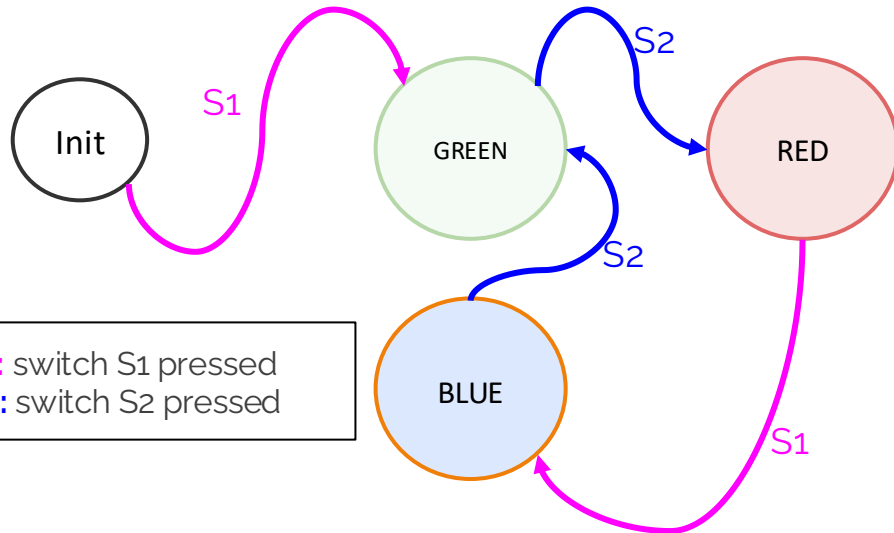
Lab#2

- When switch SW is pressed, ISR will increment count variable
 - Main code will light LEDs according to count value in binary sequence (Blue: 4, Green: 2, Red: 1)



Lab #3

- Implement the following FSM with **interrupts**:



Buttons and LEDs

